HypG: Formal Specification

Yellow Paper — Normative Algorithms, Parameters, and Contracts (v1.0)

HypG Project

September 2026

Abstract

This document normatively specifies HypG v1.0: parameters, algorithms, sizes, domain strings, API contracts, and test-vector inventory. It is the frozen companion to the implementation; in case of divergence, the frozen KAT files decide. Parameters are unchanged since v0.7 (KEM/AEAD/hash) and v0.6 (signatures). This paper makes no security claim — see the White Paper and `docs/threat-model.md`.

Contents

1 Notation and conventions **2**

2 Parameters **2**

3 Core primitives **2**

4 KEM **2**

5 Hash API **3**

6 AEAD **3**

7 Signatures **3**

8 Domain registry (frozen) **3**

9 Known-answer vectors **4**

10 API and constant-time contracts **4**

11 Test matrix (summary) **4**

1 Notation and conventions

Bytes are octets; all multi-byte integers serialize little-endian (explicit `le` conversions everywhere; no native-endian operations exist in the libraries). $\|$ denotes concatenation, $\oplus$ byte-wise xor. Security parameter: 128-bit post-quantum target (stated goal, unconfirmed). "Public" means attacker-visible (lengths, indices, counters); "secret" means key/message-derived.

2 Parameters

Symbol	Value	Meaning
n	256	ring dimension, $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$
k	2	module rank (lattice dimension 512)
q	$2^{12} = 4096$	cipher modulus (mask-only reduction)
η	2	CBD parameter, coefficients in $\{-2,\ldots,2\}$
msg	32 B	FO message (1 bit/coefficient encoding)
PK	800 B	$seed_A(32) \,\|\, \text{pack}_{12}(b)(768)$
CT	800 B	$\text{pack}_{10}(c_1)(640) \,\|\, \text{pack}_5(c_2)(160)$
SK	864 B	$seed(32) \,\|\, fake(32) \,\|\, pk(800)$
SS	32 B	shared secret
w	16	WOTS+ base; chains run all $w-1=15$ steps
ℓ_{WOTS}	67	64 message digits + 3 checksum digits
K_{FORS}	30	FORS trees; leaves per tree $2^7 = 128$
D, h	12 layers $\times$ height 5	hypertree (60-bit tree index)
SIG	35456 B	$R(32) \,\|\, \text{FORS}_{\times 30} \,\|\, \text{layers}_{\times 12}$
SIG_{pk}/SIG_{sk}	64 / 128 B	DS public/secret keys

3 Core primitives

Definition 3.1 (CA-PRF)**.** CAExpand($seed, domain, L$)*: 256-byte state from* XOF($seed \,\|\, domain \,\|$ *"-init"*)*, evolved through 32 branchless cellular-automaton rounds under a seed-derived rule, emitted as* KDF($state \,\|\, domain \,\|$ *"-out"*)$[0..L]$*. Deterministic in its inputs.*

Definition 3.2 (Hashing)**.** *BLAKE3 as specified by its designers:* H_{32} *(32 B),* XOF *(arbitrary length; first 32 output bytes equal* H_{32}*), keyed hashing, and the reference* `derive_key` *construction. No HypG modification exists.*

Definition 3.3 (Stream cipher)**.** *IETF ChaCha20 (RFC 8439 §2): 32 B key, 12 B nonce, 32-bit block counter from 0, 64 B blocks. Total input per* $(key, nonce)$ *is bounded by* 2^{32} *blocks (*2^{38} *B); counter exhaustion fails closed and never wraps.*

4 KEM

Definition 4.1 (KeyGen)**.** *On* $seed \in \{0,1\}^{256}$*:* $seed_A \leftarrow$ CAExpand($seed, seed\text{-}a$)*;* $A \leftarrow \text{Uniform}(seed_A)^{2\times 2}$*;* $s, e \leftarrow \text{CBD}_2$ *via* `CA-PRF`*;* $b \leftarrow As + e$*;* $pk \leftarrow seed_A \,\|\, \text{pack}_{12}(b)$*;* $fake \leftarrow$ CAExpand($seed, fake$)*;* $sk \leftarrow seed \,\|\, fake \,\|\, pk$*.*

Definition 4.2 (Encaps)**.** *On* (pk, m)*,* $m \in \{0,1\}^{256}$ *caller coins:* $(r, e_1, e_2) \leftarrow G(m, pk)$ *(FO oracle via* `CA-PRF`*);* $c_1 \leftarrow A^T r + e_1$*,* $c_2 \leftarrow b^T r + e_2 + \text{Encode}(m)$ *(1 bit/coefficient, spacing 2048);* $ct \leftarrow \text{pack}_{10}(c_1) \,\|\, \text{pack}_5(c_2)$*;* $ss \leftarrow \text{KDF}(m \,\|\, ct \,\|\, \text{H}(pk))$*.*

Definition 4.3 (Decaps)**.** *Decrypt* $v = c_2 - s^T c_1$*, decode to* m'*; re-encrypt to* ct'*;* $ok \leftarrow [ct' = ct]$*; return* $\text{ct_select}(ok, ss(m'), ss_{\text{rej}}(fake, ct))$*. Never fails; wrong ct yields the pseudorandom reject secret.*

Failure analysis. Decode threshold ± 1024 against noise of standard deviation ~ 50: analytic margin $\approx 20\sigma$. Empirically 20000/20000 clean round-trips (plus 100000 pre-migration on identical lattice code). The 32-trial unit test is a tripwire, not the argument.

5 Hash API

hash32 / hash64 (XOF-64, first half equals hash32), keyed32, derive32, expand (XOF($seed \parallel domain$)), kdf (XOF($domain \parallel$ parts)), and a streaming hasher proven equal to one-shot at every split pattern. All lengths are public.

6 AEAD

Definition 6.1 (One-shot seal/open)**.** *Subkeys* $(k_e, k_m) \leftarrow (\text{derive}(symm\text{-}enc), \text{derive}(symm\text{-}mac))$ *from the 32 B master key.* $ct \leftarrow \text{ChaCha}(k_e, nonce, 0, pt)$ *(in place,* $|ct| = |pt|$*); tag* $\leftarrow \text{KeyedBLAKE3}(k_m, nonce \parallel ad \parallel ct \parallel \text{le64}(|ad|) \parallel \text{le64}(|ct|))$*. Open recomputes and full-scan compares the tag* before *decrypting; mismatch (or length mismatch) returns* $\perp$ *leaving pt untouched (streaming: zeroized).*

Proposition 6.1 (Streaming identity)**.** *For every chunking of the same* $(key, nonce, ad, message)$*, the streaming sealer emits the identical* (ct, tag) *as one-shot seal: subkeys, MAC input order, and the ChaCha counter sequence (with cross-chunk partial-block buffering) coincide byte-for-byte. Tested across adversarial chunkings including empty chunks (unit) and 20 PRNG split shapes plus 1 MiB (redteam S14).*

Opener plaintext is *tentative* until `finish` validates the tag (EtM consequence); validity holds only after `Some`.

7 Signatures

Hedged $R \leftarrow \text{PRF}(sk.\text{prf}, m)$; $md \leftarrow \text{H}(R \parallel pubseed \parallel root \parallel m)$ (64 B, streamed, counter-bound); split into 30×7-bit FORS indices, 60-bit tree index, 5-bit leaf. FORS signs per-tree leaves; the hypertree climbs 12 XMSS layers (WOTS+ sign + auth path each); serialization is $R \parallel \text{FORS}_{\times 30} \parallel \text{layers}_{\times 12}$ with zeroed 8-byte reserved pads per layer, rejection-checked at verify. Verify returns false on any malformation (length first). Composition bounds are cited in `docs/spec-ds.md` §6 (closed-form FORS/WOTS+/XMSS reductions); auditor must confirm on these exact numbers.

8 Domain registry (frozen)

Domain (`HYP-G/v07/` prefix)	Purpose
`seed-a`	KEM $seed_A$
`sec-s`, `sec-e`	KEM secret / error vectors
`poly-a`	KEM matrix A
`rand-r`, `rand-e1`, `rand-e2`	FO randomness G
`fake`	implicit-rejection key
`r-seed`	FO randomness seed
`ss`, `reject`	shared-secret / rejection KDFs
`symm-enc`, `symm-mac`	AEAD subkeys
`ds-pubseed`, `ds-prf`, `ds-addrnd`, `ds-msghash`	DS derivations
`kat`	hash-KAT sample only (never protocol use)

Frozen rows are never edited; new domains require registry rows. `CA-PRF -init/-out` suffixes are mechanism, not registry.

9 Known-answer vectors

File	Contents	Anchors
`kats/v0.7.0.txt`	KEM, 3 seeds	BLAKE3("abc")
`kats-symm/v0.7.0.txt`	AEAD, 5 cases	empty/edge/1000 B
`kats-ds/v0.7.0.txt`	DS, 2 seeds × 2 msgs	empty + 55 B
`kats-hash/v0.7.0.txt`	hash32/64, keyed, derive	empty/abc/1025 B

Byte-exact gates compare renderer output to these files; history (v0.4–v0.6) is retained. Empty/abc/XOF-64 anchors were cross-checked against third-party BLAKE3 bindings at freeze time.

10 API and constant-time contracts

Public Rust API: `keygen/encaps/decaps` (+ key/ciphertext/secret types), `hash32/hash64/keyed32/derive32` `seal/open`, `StreamSealer/StreamOpener`, `ds_keygen/ds_sign/ds_verify`, `mt_sign`. All fallible APIs return `Option` (never panic); all loops have public bounds; no secret branches, indices, or early returns; lengths are public. Debug renderings of key-holding types are redacted; `Drop` wipes reachable key material (best-effort under `forbid(unsafe)`).

11 Test matrix (summary)

129 tests (96 unit, 20 CLI failure-path integration, 13 doc-tests) on stable debug/release and MSRV 1.70; redteam S1–S14 (incl. full 6400-bit malleability, 20000-trial failure sweep, 1011-case panic corpus, streaming games); lattice L1–L5; Miri clean (core/symm/KEM); Welch t-test max $|t| \approx 1.23$ at $n = 10000$ (informational). Full matrix: `docs/testplan.md`.

License: GNU General Public License v3.0 or later. Design companion: the HypG White Paper.